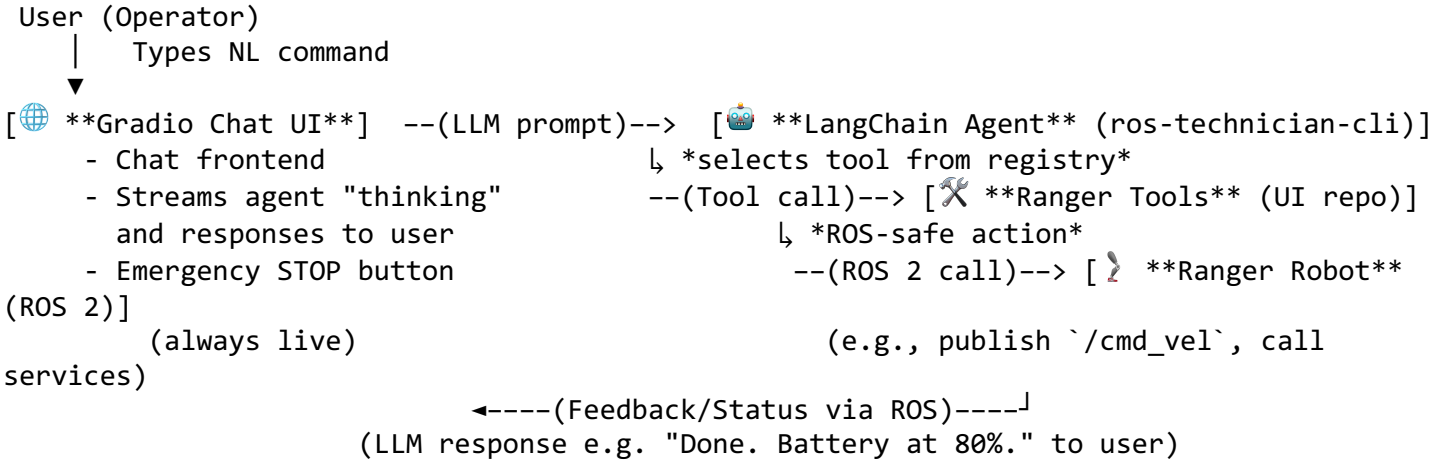
LLM-Driven ROS2 Robot Operator UI Design

System Architecture and Data Flow

Overview: The system integrates a Gradio-based chat UI with a LangChain agent (from the `ros-technician-cli` submodule) to interpret natural language commands and execute safe ROS 2 actions on the Ranger robot. Key components include the web UI, the agent (LLM + tools), ROS 2 interfaces for robot control, and safety/monitoring layers.



- **Gradio Chat UI:** A web interface (running on the robot's computer) that allows the user to type natural language commands. It displays the conversation and *the agent's tool usage steps in real-time* (using Gradio's `Chatbot` streaming)[1][2]. The UI also provides manual override controls (e.g. joystick, stop button) as a safety fallback.

- **LangChain Agent (ros-technician-cli):** This is the brains of the system. It's an agent built on LangChain (imported from the `ros-technician-cli` submodule) that interprets user instructions and decides on actions. The agent uses an LLM (either OpenAI ChatGPT API or a local model) and a set of **Tools** to fulfill requests. We **reuse the existing agent implementation** from `ros-technician-cli` and extend it with Ranger-specific tools rather than writing a new agent from scratch (ensuring we leverage its prompt, planning logic, and ROS introspection capabilities).

- **Ranger Tools (UI repo):** A **registry of vetted tool functions** representing allowable robot operations. Each tool is a deterministic function call (no free-form code execution) that wraps a specific ROS 2 action or query. For example:

- *Movement Tools*: e.g. `move_forward(distance_m)` publishes a Twist to drive forward X meters within safe limits.
- *Rotation Tool*: e.g. `turn_angle(degrees)` to rotate in place.
- *Stop Tool*: immediately stops the robot (publishes zero velocities or calls an emergency stop service).
- *Status Tools*: fetch telemetry like battery level, system health, sensor status, etc.
- *Diagnostics Tools*: check for any errors or list active ROS nodes.

This design follows a similar pattern to the TurtleBot3 natural-language agent, which organizes robot capabilities into distinct tool modules (e.g. motion, status, sensor tools)[3]. Our "Ranger tools" will be implemented in Python (using rclpy) within the UI repo, and the LangChain agent can invoke them via

LangChain's standard tool interface. Each tool is restricted to a *pre-defined ROS action/service/topic*, preventing arbitrary commands outside the allowed set. The agent's prompt will be configured to **only use these tools** for execution – if the user asks for something outside this scope, the agent should refuse or ask for clarification (mitigating hallucinations).

- **ROS 2 Interface (rclpy Node):** The tools internally use ROS 2 Humble APIs (via `rclpy` in Python) to interact with the robot. Using rclpy aligns with the rest of the stack (Python-based UI and agent), making integration easier and faster to iterate[4]. Each tool function either publishes to a topic (e.g. `/cmd_vel` for movement), calls a ROS 2 service/action (e.g. a `/stop` service, or `/get_battery` service), or reads from a topic (e.g. subscribe to `/battery_state`). We favor `rclpy` here since this UI and high-level logic benefit from Python's rapid development and easier integration with LangChain, whereas performance-critical low-level controllers remain in the robot's C++/rclcpp code[4]. (In ROS2, it's common to use Python for user-facing or prototyping tools and C++ for real-time or heavy data processing[4].)

- **LLM Provider Abstraction:** The agent's LLM backend is modular. We design a provider interface so that we can plug in either Option A: OpenAI's ChatGPT via API, or Option B: a local model (e.g. served by Ollama) without changing the agent logic. For example, we might use a library like **LiteLLM** or a custom wrapper that presents a unified `generate()` interface. In practice, we'll allow configuration of the model via environment variables or a config file (as seen in similar projects, e.g., the TurtleBot3 agent allows setting `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, etc., and choosing the model name in a config param)[5]. Our agent initialization will check a setting like `LLM_PROVIDER` to either instantiate LangChain's `ChatOpenAI` (for OpenAI) or a local inference model (for example, via `ollama` or HuggingFace pipeline). This ensures the UI can run with an internet connection (ChatGPT) or fully offline with a decent local model.

- **Logging & Monitoring:** All agent decisions and tool calls generate structured logs. Each time the agent invokes a tool, we log a JSON command record (including timestamp, tool name, parameters, and result) to a file or database. The UI can display a *command history* to the user, and these logs enable post-hoc analysis or an "Explain what the robot did" feature (the agent could be asked to summarize the log in natural language). For transparency, the Gradio chat also visually shows when a tool is used (e.g., "🤖 Used tool *MoveForward*") along with the agent's thoughts[6]. This uses Gradio's ability to show intermediate steps in a collapsible format for the user[7].

- **Safety Controller:** The architecture includes safety interlocks at multiple levels:

- A **persistent STOP/E-Stop channel** that bypasses the LLM: e.g., a big red "STOP" button in the UI that directly calls the Stop tool or a dedicated ROS topic to halt the robot immediately. This is always available regardless of agent state.
- The tools themselves enforce safety constraints – for instance, the Move tool will cap speed and acceleration to known safe values (never exceeding the robot's limits). It will also check context (e.g., if a movement distance is too large or a path is not clear, it might require confirmation).
- **Confirm-before-execute:** For high-impact or non-trivial commands (like "navigate to the shed" in future navigation mode), the agent will pause and ask the user for confirmation. Implementation-wise, we can have certain tools or plan steps flagged as "requires_confirmation". The UI can then prompt "💡 The robot is about to navigate 10 meters to the shed. Proceed? (yes/no)". Only on yes will the action be actually dispatched to ROS. This prevents the robot from wandering off on a potentially unsafe action based solely on an LLM suggestion.

- The agent's prompt will be constrained to *refuse or clarify any command that is not supported by an available tool*. It will not execute arbitrary code or OS commands. By using a closed set of tools and structured outputs (akin to function calling), we achieve deterministic execution paths – the LLM can **only effect change through our tools**. (For example, if the user asked to "self-destruct" the robot, there is no such tool, so the agent will safely respond it cannot do that.)
- **Time-outs and error handling:** Each tool invocation will have a timeout and retry policy. If a ROS action does not complete or a topic publish times out (no confirmation), the tool will return an error result to the agent (and the agent can relay that to the user as "I couldn't complete that action."). The system monitors the ROS 2 execution – if the robot becomes unresponsive (e.g., ROS node crashes or lost connectivity), the UI will notify the user and suggest switching to manual mode.

- **Manual Teleop & Fallback:** In addition to chat, the UI offers a basic dashboard for direct control. For MVP, this could be simple arrow buttons or joystick (using Gradio components or a small JS widget) that publish `Twist` commands to move the robot. This serves two purposes: (1) a fallback if the LLM agent is uncertain or if the user prefers precise manual control in a situation, and (2) a way to teleoperate the robot if needed (e.g., driving it to a safe spot). The UI will also display key telemetry (battery percentage, etc.) continuously so the user has situational awareness.

Rationale: By separating concerns – UI, agent decision-making, and ROS tool execution – we ensure modularity. The UI focuses on user interaction and safety overrides, the LangChain agent handles language understanding and task planning, and the ROS tools perform only allowed low-level actions. This matches patterns seen in other ROS2+LLM projects: for example, the TurtleBot3 agent mentioned above cleanly splits movement, status, and sensor operations into different tools[3]. Our design effectively turns high-level user requests into structured robot actions with multiple checkpoints for safety and user confirmation.

Repository Layout (New Repo: `ranger-llm-ui`)

Below is a proposed file tree for the new repository, **tentatively named `ranger-llm-ui`** (or `ranger-operator-ui`), which will house the Gradio UI and ROS interface:

```
ranger-llm-ui/
├── README.md
├── package.xml                       # ROS2 package manifest for colcon
├── setup.py, setup.cfg, pyproject.toml   # Python package setup for rclpy node
├── ranger_llm_ui/                    # Python package directory (ROS2 node code)
│   ├── __init__.py
│   ├── ui_node.py                   # Main entry: starts rclpy and Gradio app
│   ├── agent_interface.py           # Integrates LangChain agent (ros-technician)
│   ├── tools/                       # Directory for Ranger tool implementations
│   │   ├── __init__.py
│   │   ├── movement_tools.py        # e.g. Move, Turn, Stop tools (calls ROS
topics/services)
│   │   ├── status_tools.py          # e.g. BatteryStatus, NodeHealth tools (calls ROS
services or topics)
│   │   ├── … (other tool modules as needed) …
│   │   └── all_tools.py             # Registers all tools for the agent
│   ├── schemas/                     # Define JSON schemas or dataclasses for command
formats (if used)
│   │   └── commands.py              # E.g., dataclass definitions for each command type
│   ├── safety/
```

```
│   │   └── guard.py               # Safety checks (velocity limits, confirmation logic)
│   └── utils/                     # Utility modules (logging, config loading, etc.)
│       └── logger.py              # Structured logging of commands
├── ranger-garden-assistant-ui.launch.py   # Example ROS2 launch file to start the UI
node
├── ros-technician-cli/            # **Git submodule** containing ros-technician-cli
│   └── ... (contents of the ros-technician-cli repository) ...
└── tests/                         # (optional) Test scripts for tools
    └── test_tools.py
```

Notable elements:

- The repo is a *ROS 2 package* (with `package.xml` and possibly an ament setup) named something like `ranger_llm_ui`. This allows it to be built with colcon and run with `ros2 run` or via launch files. Inside, the Python code is organized in a way familiar to ROS 2 Python packages (similar to how one might structure a rclpy node package)[8].

- We include **`ros-technician-cli` as a git submodule**. This will appear as a folder `ros-technician-cli/` within our repo, but it actually points to the external repository. We will use it as a library. For instance, in `agent_interface.py` we might do:

  ```
  from ros_technician_cli.agent import ROSA  # hypothetical ROSA base class
  ```

  or import a specific agent class if available (maybe `from ros_technician_cli import TurtleAgent`). We will not modify the submodule's code; instead, we extend or wrap it. This approach ensures we can pull upstream updates to the LangChain agent logic easily.

- **Gradio integration:** The `ui_node.py` will initialize the ROS 2 node (rclpy), set up subscriptions (if any, e.g. to battery topics), then launch the Gradio Blocks interface. It will use Gradio's async features to allow the agent to stream responses. For example, it will define an async handler similar to the LangChain+Gradio example[1][2]: calling `agent.astream()` and yielding intermediate tool-use messages and final output to the UI. Because Gradio can coexist with other event loops, we ensure rclpy spinning doesn't block the UI – likely by running the ROS spin in a separate thread or using `rclpy.spin_once` in the Gradio loop. (Alternatively, since the UI runs on the robot, using rosbridge is unnecessary overhead; direct rclpy calls are simpler and more efficient.)

- **Tool definitions:** Each tool module in `tools/` defines one or more LangChain Tool objects. For instance, `movement_tools.py` might have:

  ```
  from langchain.agents import tool
  @tool("MoveForward", return_direct=True)
  def move_forward(distance: float):
      \"\"\"Move the robot forward by a specified distance in meters.\"\"\"
      # (calls ROS service or publishes Twist)
      return "Moving forward %.1f meters" % distance
  ```

  We will consolidate them in `all_tools.py`, which constructs a list of all Tool objects for the agent. The `agent_interface.py` then loads these tools into the agent (e.g., `agent = ROSA(tools=all_tools, llm=llm_instance)` or if ROSA base doesn't take tools list, we might subclass ROSA to inject our tools).

- **Schemas (commands.py):** While LangChain tools typically take free-form string input, we can define structured command representations for clarity and logging. For example, a JSON schema for a move command:

  ```
  { "action": "move", "direction": "forward", "distance": 1.0 }
  ```

  or for a turn:

  ```
  { "action": "turn", "angle": 90 }
  ```

  These schemas can be used by the tools to validate inputs (and by the agent prompt to format its intended actions). In a future iteration, we might even utilize function-calling capabilities of the LLM to ensure it outputs a JSON that matches these schemas[9]. For MVP, we will likely keep it simpler: the agent will output a text like "use MoveForward 1.0" and LangChain's parser will map that to our tool.

- **Safety guard module:** `safety/guard.py` will contain logic for sanity-checking commands before execution. The tool functions will call these utilities. For example, before executing a move, `guard.py` can clamp the requested speed/distance to a maximum, and if a command is high-risk (e.g., a very long move), it can require a confirmation flag from the agent/UI.

- **Launch and entry point:** We provide a ROS 2 launch file (e.g. `ranger-garden-assistant-ui.launch.py`) that can be included in the main robot's launch system. This would start the `ui_node` with appropriate parameters (like which LLM provider to use, or what ROS namespace to operate in). The entry point for the package (in setup.py `console_scripts`) will map to something like `ranger_llm_ui.ui_node:main`, so it can be invoked via `ros2 run ranger_llm_ui ui_node`.

Submodule Integration & Build Process

To set up the submodule and workspace:

- **Adding ros-technician-cli Submodule:** In the new repo, we run:

  ```
  git submodule add https://github.com/anh0001/ros-technician-cli.git ros-technician-cli
  git commit -m "Add ros-technician-cli submodule"
  ```

  This will include the submodule at a specific commit (it's wise to pin a known-good revision of `ros-technician-cli`). In the README, we'll instruct developers to init/update submodules (`git submodule update --init`) when cloning.

- **Inclusion in Ranger workspace:** The main robot repository (`ranger-garden-assistant`) will include this new UI repo as a submodule as well. For example, within `ranger-garden-assistant`, we might add it under an `external/` or `ui/` directory:

  ```
  cd ranger-garden-assistant
  git submodule add https://github.com/anh0001/ranger-llm-ui.git ui/ranger-llm-ui
  git commit -m "Add LLM UI submodule"
  ```

  Now, the `ranger-garden-assistant` repo has a pointer to the UI repo. If `ranger-garden-assistant` is itself a colcon workspace (with a `src/` folder), we ensure the submodule is placed

accordingly. It likely already has a `src/` containing various packages – we can place `ranger-llm-ui` alongside them so that a single `colcon build` builds everything.

- **Workspace Overlay vs Single Workspace:** Since the UI will run on the same machine as the robot, a single combined workspace is simplest. We will treat `ranger-llm-ui` as just another package in the robot's workspace. The alternative is an overlay (having the UI in a separate workspace that overlays the robot's install) – but that adds complexity in launch and coordination. The recommendation is:

- Add the UI package to the robot's workspace.
- Update `ranger-garden-assistant` documentation or launch to optionally launch the UI. The UI can be started/stopped without affecting core robot function aside from ROS messages.

Building: The UI contains both Python code and the C++/Python code from submodules if any. Colcon should detect `package.xml` in `ranger-llm-ui` and build it (which for Python means just installing it). The submodule `ros-technician-cli` might itself be a pip package rather than a ROS package. If so, we treat it as a vendor library – possibly adding its path to `PYTHONPATH` or installing it via `pip install -e ros-technician-cli` in the build process. We might automate that in the `setup.py` or via a colcon hook. For the MVP, a straightforward approach is to add `ros-technician-cli` as a dependency in `requirements.txt` or `setup.py` (pointing to the local submodule directory). This way, when building our package, it installs the submodule as well.

- **Pinning Versions:** We will pin the submodule to a specific commit (documented in a `VERSIONS.md` or the README). This ensures that updates to `ros-technician-cli` are controlled. When we want to update it, we manually bump the submodule pointer and test compatibility.

- **Running:** After building, the UI can be launched via ROS launch system. For example:

  ```
  ros2 launch ranger_llm_ui ranger-garden-assistant-ui.launch.py
  ```

  This could start our `ui_node` (and we can include parameters like `agent_model:=gpt-4` or `use_ollama:=true` etc.). Alternatively, one could run directly:

  ```
  ros2 run ranger_llm_ui ui_node
  ```

  and it would start the Gradio server, logging into ROS as a node (so it can publish/subscribe).

- **Networking:** Because this is on the same machine, ROS 2 communication is direct (no rosbridge needed). The Gradio UI by default will bind to localhost (and possibly to network IP if we want LAN access). Since the operator may access via a browser, we ensure proper network config. (No special ROS discovery issues since it's local; if accessed from a remote browser, just need to expose the Gradio port.)

- **Development Note:** We will include instructions to run the UI in isolation (for development) and inside the full robot system. Possibly we'll allow a "dry-run" mode where the UI can run without a robot, using a simulated ROS (or simply not executing actual moves – useful for testing the LLM responses). This can be done by mocking the ROS interfaces when no ROS master is detected.

Interface Contracts and ROS Mapping

We define a clear **command schema** for all robot actions to ensure consistency between the agent, tools, and ROS interfaces. This schema can be represented as Python dataclasses or JSON structures.

Command Types:

1. **MoveCommand** – Moves the robot linearly.
2. Fields: `direction` (enum: forward/backward), `distance_m` (float, meters).
3. Implementation: Calls a ROS 2 Action `/drive_distance` if available, or publishes on `/cmd_vel` with a timed controller. If using a Twist publisher, the tool will publish a forward velocity and sleep until the distance is covered or an obstacle is detected. Velocity is chosen based on safe limits.
4. Example: `{"action": "move", "direction": "forward", "distance_m": 1.0}` leads to a forward motion of 1.0m.
5. **TurnCommand** – Rotates the robot in place.
6. Fields: `angle_deg` (float, positive for clockwise or such convention).
7. Implementation: Calls a `/rotate_angle` service or publishes angular `Twist`. Ensures rotation speed limit.
8. Example: `{"action": "turn", "angle_deg": 90}` (rotate 90° right).
9. **StopCommand** – Immediate stop.
10. Fields: none (or maybe reason if any).
11. Implementation: Publishes zero velocities to `/cmd_vel` and/or calls an emergency stop service on the motor controller. Also cancels any ongoing Move/Turn action. This tool will be called either by user pressing STOP or by the agent if it detects a need to halt.
12. This is always available (even if not explicitly invoked by the agent, the UI's STOP button triggers it).
13. **StatusCommand** – Fetches overall status.
14. Could be broken down into sub-tools:
    - **BatteryStatus**: reads a `/battery_state` topic or calls `/get_battery` service, returns battery % or voltage.
    - **SystemHealth**: uses ROS 2 introspection – e.g., checks if all essential nodes are running and topics are flowing (could reuse diagnostic data or the ros-technician's own tools). The `ros-technician-cli` agent likely already has tools for listing nodes, topics, etc., which we can reuse[10]. We might just call those from our agent for "diagnostics" queries.
    - **SensorStatus**: e.g., query if camera or lidar is publishing (could subscribe to those topics or use ROS2 topic info).
15. These are read-only tools that return data for the agent to relay. Example user query: "What's your battery level?" -> agent invokes BatteryStatus tool -> gets "Battery at 78%" -> agent replies to user.
16. **NavigationCommand** (Future scope): High-level goal navigation.
17. Fields: `destination` (could be a named location or coordinates).
18. Implementation: This would interface with the robot's navigation stack (e.g., send a goal to Nav2 action server). Because this is complex and needs map coordinates, it will be implemented in future iterations with heavy safety gating (always confirm with user and perhaps break into subtasks).

19. **Other Utility Commands:** e.g., *TakePhoto*, *Scan* (if robot has camera or lidar and user wants data). These would trigger sensor data captures. For MVP, we might not include these unless the hardware is present and user needs it, but the framework allows adding them easily.

Tool Invocation via Agent: In LangChain's standard agent loop (ReAct pattern), the LLM will output something like:
"`Action: MoveForward\nAction Input: 1.0`"
The agent framework will then call our `move_forward` tool with argument `1.0`. Our implementation will parse that (or use LangChain's argument schema) and execute the MoveCommand. We restrict the agent to known actions by constructing the prompt with tool names and descriptions only. The LLM thus knows it can only respond with one of those actions when it needs to act.

To make tool use deterministic and safe, we might leverage LangChain's support for **OpenAI function calling** or a similar mechanism in the future. For now, a well-crafted prompt and the closed tool list provide the boundary.

ROS Topic/Service Mapping Examples:

- MoveCommand -> ROS 2: e.g. publish `geometry_msgs/Twist` on `/cmd_vel` at say 0.2 m/s forward until distance achieved. Alternatively, if `ranger-garden-assistant` provides a service `/move_distance` that takes a distance and handles the motion (including obstacle avoidance if any), use that for more reliability. We will confirm the robot's API: since it's said teleop primitives are stable, possibly there is an existing *teleop node* that we can interface with. If not, implementing a simple distance controller in the tool is acceptable for MVP.

- TurnCommand -> ROS 2: similarly, publish angular velocity on `/cmd_vel` or call a `/rotate` service if available.

- StopCommand -> ROS 2: publish zero on `/cmd_vel` (perhaps multiple times) and call any low-level brake if available. Ensure this is high-priority (maybe use a separate ROS node handle with QoS to guarantee delivery).

- BatteryStatus -> ROS 2: subscribe to `/battery_state` topic (sensor_msgs/BatteryState) and return the percentage. The tool might simply read the last message from that topic (the `ui_node` can keep a subscription updating a variable).

- NodeHealth -> ROS 2: this could use the ros-technician's capability: for example, `ros-technician-cli` (or NASA's ROSA) has tools to list nodes or topics[10]. We might call those functions directly. Or use `rclpy` APIs to get node graph info (like which topics are publishing). For MVP, we can implement a simple check: e.g., ensure critical topics (like odometry, sensor topics) are active by checking their publication rate via `rclpy` topic info. If anything is off, report it.

Communication Patterns: We prefer ROS 2 services or actions for discrete tasks like moving a certain distance (so we know when it's done). If those aren't available, our tools will implement their own monitoring (e.g., subscribe to odometry and compute when the distance is reached, then stop). Tools will run in the background of the agent call – for long actions, we might have the agent wait or periodically check progress (LangChain tools can either return immediately or stream results; we may just block the agent until action is done or a timeout triggers).

Example Interaction:
User: *"Move 2 meters forward and report battery."*
Agent (LLM decides plan): First uses Move tool -> our MoveCommand tool executes, moving the robot

~2m. Once done (or even during movement), agent then calls BatteryStatus tool -> gets battery %. Finally, agent responds: *"✅I moved 2 meters forward. The battery is at 75%."*.
Throughout this, the UI would show something like: "🤖 *Used tool MoveForward*" (with any log info) then "🤖 *Used tool BatteryStatus*", before the final answer[6].

By adhering to a defined set of actions and corresponding ROS calls, we ensure the LLM cannot invoke undefined behavior. This approach mirrors that of other ROS LLM interfaces – for instance, the TurtleBot3 agent's demo of moving in a square involved the agent invoking motion tools step by step[11][12]. We will achieve similar controlled multi-step execution through the LangChain agent's planning.

Feature Roadmap

We plan the development in stages:

MVP Feature Set (Immediate Implementation)

- **Natural Language Commanding:** Users can type commands like "move forward 1 meter", "turn right 90 degrees", "stop", "what is your battery level?", etc. The system executes them using the appropriate tools and responds in chat. Basic movements (forward/back, turn) and status queries are supported out of the box.

- **Gradio Chat Interface:** A simple chat UI with streaming responses. The agent's thought process (tool selection) is visible to the user for transparency (as collapsible messages titled "Used tool X")[6]. This helps build user trust and allows intervention if the agent's plan looks wrong.

- **Core Tools Implemented:** MoveForward, MoveBackward, Turn, Stop, BatteryStatus, SystemStatus. These cover the primary use-cases: driving the robot and checking its state. Each tool is tested in simulation or on the robot for correct behavior. They have built-in parameter validation (e.g., distance must be positive and within some range).

- **Safety Mechanisms:** The emergency STOP button in the UI is fully functional – it overrides any ongoing action. Velocity and distance limits are enforced (e.g., even if user asked for 100 m, the tool might require confirmation or split it into smaller safe segments). The agent is prompt-engineered not to do anything outside provided tools, and to confirm dangerous requests. Timeouts for motion commands ensure the robot doesn't get stuck moving indefinitely.

- **Basic Command History/Logging:** The UI will display recent commands (perhaps as a sidebar or just scrollback in chat). Additionally, we log to a file each command and outcome. This will help with debugging and future "explain" features.

- **ROS2 Node Integration:** The UI runs as a ROS2 node (so it can easily share context with other ROS nodes). It can subscribe to at least the battery topic to get live updates. (We may also show the battery in the UI header continuously, aside from chat responses.)

- **One LLM backend initially:** MVP will likely use OpenAI GPT-4/GPT-3.5 via API (assuming connectivity) because of reliability. We will, however, structure code to allow easy switching to a local model (perhaps include a config flag and dummy local model that echoes, to prove the interface). Full local model integration might come in next phase once we test some (like Code Llama or an instruct model via Ollama).

Near-Term (Next ~4 Weeks) Planned Features

- **Expanded Toolset (Task Automation):** Introduce tools for *multi-step tasks*. For instance, a **SequenceTool** that can execute a series of primitives (if the user says "move in a square 0.5m side", the agent currently would plan 4 moves and 4 turns; we can handle that via the agent's reasoning or add a convenience tool). Another example: **DockingTool** (if the Ranger has a docking station) could be added. We'll also incorporate any Ranger-specific actions (e.g., water plants if it has an arm, etc.) as needed by the domain – these would be carefully programmed and gated.

- **Enhanced UI:**

- A panel for **Command History** with timestamps and statuses (success/fail).
- A panel for **Robot Telemetry Dashboard** showing live data: battery, speed, maybe a basic lidar scan visualization or camera feed if relevant. (For example, showing a camera feed can reuse ROS's `web_video_server` in a Gradio image component, or a minimal custom integration.)
- **Confirmation Dialogs:** Implement the confirm-before-execute flow in the UI. If the agent outputs an action that is marked "pending user confirmation", the UI will present a Yes/No choice instead of executing immediately. The agent will wait (we might implement this by pausing in the tool and not returning until user input is received).

- **Structured Logs & Replay:** Improve logging to record not just text but the actual JSON commands and responses. Possibly develop a small utility to replay a session or feed it to the agent for explanation. We might allow the user to click on a past command in history and ask "why did you do this?" – the agent can then explain its reasoning (since LangChain can be prompted with its own thought log).

- **Multiple LLM Backend Support:** Finish the integration of a local model through the provider interface. For example, hooking up to an **Ollama** endpoint running Llama2 or similar. We'll abstract LangChain's LLM class such that switching is a config change (OpenAI's GPT vs local model). We will test with a smaller model to ensure the agent still functions (though possibly with reduced reliability). We may consider using **LangChain's support for local models** or direct calls to something like `ollama` CLI. The user will be able to choose backend in a config file or environment variable.

- **Error Handling & Resilience:** Make the system robust to common failures. E.g., if the LLM returns a malformed action (not matching any tool), we catch it and ask the user for clarification. If a ROS service call fails (maybe the robot's arm is not responding), the agent informs the user gracefully. We'll also handle Gradio UI disconnects (if the browser refreshes, the session can possibly restore state or at least the ROS node continues running).

- **Testing in Simulation:** By week 4, we aim to have tested the UI and agent extensively in a simulated environment (e.g., using Gazebo or at least on a TurtleBot or Ranger simulation). This ensures that the motion commands do what we expect and that the safety features work under various conditions. Any edge case where the LLM might hallucinate an unsupported action will be addressed via prompt or code adjustments.

Future (Long-Term) Ideas

- **LLM-Assisted Navigation & Complex Tasks:** Allow the user to give high-level goals like "inspect the garden and report if any plants need water". This would require the agent to plan a complex sequence (navigate to multiple points, perhaps take pictures or read sensor data, analyze it, etc.).

We will integrate with the robot's navigation stack (Nav2) via a **NavigateToLocation** tool. This tool will definitely require user confirmation or even supervised autonomy (the agent proposes a route or set of waypoints, and the user approves). We will also integrate semantic mapping – e.g., the robot's known locations or an object detection model to let the agent reason about "plants" or other domain entities. Strict safety gating will be in place: navigation actions won't execute without explicit user go-ahead, and possibly real-time monitoring for obstacles.

- **Vision and Manipulation Integration:** If Ranger has a camera or arm, future versions can incorporate vision-language actions. For example, "pick up the yellow tool from the table" would require the agent to identify the object (via a vision model or predefined location) and then call manipulation primitives. This would entail adding tools like **DetectObject** (calls a CV model) and **MoveArm** (calls a MoveIt motion plan). Each of those in turn are heavily validated for safety (collision checking, etc.). This is beyond MVP, but the architecture is extensible for that – new tools can be added in the registry with minimal changes to the UI or agent core.

- **Voice Interface:** Gradio can support audio input; in future we could allow voice commands from the operator, converting speech to text (using an STT service) and feeding to the agent, then using TTS for the agent's reply. This would make the interface more hands-free in the field.

- **Multi-Modal Feedback:** Show a map or location of the robot in the UI. After navigation, the agent could present a map with the route taken, or images captured. Essentially, the agent could gather data via tools and the UI can display it (rich media responses).

- **Collaboration & Learning:** Over time, the agent could learn from corrections. We might integrate feedback mechanisms: if the user rephrases or corrects the robot's action, the system could adjust the prompt or fine-tune a local model. We may also incorporate a knowledge base (documents about the robot and environment) that the agent can use to answer questions or explain its decisions (Retrieval-Augmented Generation).

Throughout these expansions, **safety remains the top priority**. Especially for full autonomy tasks, incorporating something like a "safety supervisor" (perhaps a secondary agent or rule-based checker monitoring the LLM's plans) could be wise. For example, an AI watchdog that ensures no navigation command is issued without a recent map or that checks if the LLM output aligns with known safety rules[13].

Safety and Failure Mode Analysis

Even with careful design, various failure modes are possible. We analyze them and outline mitigations:

- **LLM Hallucination or Misinterpretation:** The LLM might misunderstand a command or hallucinate an unavailable tool. *Mitigation:* We explicitly restrict the agent's action space to the tool set. The prompt given to the LLM will include instructions like *"If the command is not one of [list of operations], do not invent new actions."* If the agent still produces an unknown action, the agent code will catch it and respond with an error or request clarification from the user instead of executing. We also use high-quality models (GPT-4) for better reliability in following the tool-use format, and test with many phrasing variations.

- **Stale World Model / Context:** The agent might not have up-to-date info (e.g., battery level or an obstacle) when planning an action, leading to a poor decision. *Mitigation:* We incorporate real-time queries in the chain. For example, before moving, the agent (or a pre-check in the tool) could query a "safe to proceed?" sensor check (like a simple collision avoidance sensor status). We

also allow the user to intervene: since the UI shows what the agent is about to do, the user can hit STOP if they realize it's not right. Moreover, for any significant action, the agent can be prompted to double-check recent sensor data ("CheckStatus" tool) before execution. In future, we could maintain a shared blackboard of recent state that the LLM can automatically examine.

- **ROS Communication Failures:** The robot might lose connection (ROS node crash or DDS network issue). For instance, a movement command might not actually reach the robot. *Mitigation:* The tool calls will have timeouts. If a movement action doesn't start (no odometry change) within say 2 seconds, the tool returns an error and the agent tells the user "I couldn't move, please check the robot." The UI will also monitor ROS events – we can subscribe to `/rosout` or listen for node heartbeat. If the whole ROS system is down, the UI can display "⚠️ Lost connection to robot" (since our UI is running on the same machine, this likely means a critical failure; the user might need to restart things).

- **Physical Safety – Collision or Tip-over:** If the robot is told to go somewhere unsafe (like off a step or into an obstacle), the lower-level robot controllers should ideally prevent it (e.g., Ranger may have obstacle sensors or we integrate with Nav2 which has obstacle avoidance). But assuming simple teleop, the risk exists. *Mitigation:* Very conservative movement: for MVP, we treat all movement as **line-of-sight teleop** – essentially remote control with the user supervising. The user should only command what they can see is safe. We will however add in our documentation that this system is not fully autonomous navigation (until we integrate Nav2). In near-term updates, we'll integrate with Nav2 for path planning, which has obstacle avoidance. Another mitigation: enforce small step sizes (e.g., agent should move in increments like 0.5m and then re-evaluate). The agent, seeing camera data in the future, could decide to stop if an obstacle appears. We also rely on the user's common-sense: that's why confirm big moves.

- **Emergency Scenarios:** If something goes wrong (robot not responding properly, or moving incorrectly), the UI's STOP sends an immediate zero-velocity command. We will make this a direct `rospy.Publisher` latched at high rate for a second to ensure it overrides any other command. Also, if the agent for some reason gets into a loop or long output, the user can always refresh or kill the UI – the robot won't do anything without an explicit tool command.

- **Latency and Timing Issues:** LLM API calls introduce latency. A user might say "stop now" but the agent takes 5 seconds to respond due to LLM latency. *Mitigation:* The STOP button bypasses LLM as mentioned. For voice or textual "stop" commands, we can implement a special-case: if the user message is simply "stop" or "emergency stop", we don't route it through the LLM at all – we directly execute StopCommand in the backend before the agent even thinks. This ensures immediate reaction. Another approach: run a lightweight keyword spotter if voice commands are used, to catch "stop" even faster.

- **Tool Execution Errors:** A tool might throw an exception (e.g., service call fails, or sensor data not available). *Mitigation:* We wrap each tool call in try/except. On error, the tool returns a message like "<ToolName> failed: reason", which the agent will convey. We will also log the stack trace to debug. Over time, we'll harden the tools for robustness (for example, if battery topic hasn't published yet, we could wait a moment or use last known value).

- **Concurrency and Multi-step Consistency:** The agent might issue a new command while a previous one is still executing (though LangChain agents typically wait for one tool to finish). In case of long actions, the agent might "think" it's done when it's not. *Mitigation:* We ensure our tools are mostly synchronous from the agent's perspective – e.g., a MoveDistance tool will not

return until the move is completed (or timed out). This way, the agent will not start the next step too early. If we do need concurrency (like monitoring something while moving), we'd handle that carefully (maybe splitting the agent tasks or using threading at tool level). But MVP will avoid that complexity – one command at a time.

- **User Interface Disconnect:** If the browser is closed, the Gradio server keeps running on the robot. The ROS node continues to function and can still respond to ROS events (though no one is viewing it). When the user reconnects, they might start a fresh session – we might not yet support persistent chat history after reconnect (unless we save it). As a mitigation, critical things (like if the robot was in middle of a task) should either continue to completion or safely stop if no confirmation given. We will document that the UI needs to remain open for supervision.

Overall, we take a **defense-in-depth** approach to safety: the LLM is constrained by design, the tools have safety checks, and the user is kept in the loop for supervision. This is in line with recommended practices for AI in robotics (never fully trust the AI, always have an override)[13].

Implementation Plan and Milestones

1. **Project Setup (Week 0-1):** Create the `ranger-llm-ui` repository and set up basic structure. Add `ros-technician-cli` as a submodule (point to a tested commit). Write a minimal ROS2 node that can launch Gradio (just a placeholder UI) to verify the colcon build and launch process. *Milestone:* Robot can run `ros2 run ranger_llm_ui ui_node` and see a hello-world webpage.

2. **Integrate LangChain Agent (Week 1):** In `agent_interface.py`, instantiate the `ros-technician-cli` agent. For initial tests, use a dummy LLM (or OpenAI API with a test key) and a trivial tool (like a Ping tool that just returns "pong"). Ensure we can call `agent.invoke("ping")` and get a response. Then modify the Gradio UI to pipe user input to `agent.invoke` or `agent.astream`. Verify that streaming works – e.g., for a question that triggers tool use, see intermediate steps in the UI[6]. *Milestone:* User can ask something like "what topics are active?" and the agent (using ros-technician's existing tools) responds with a list (text-based, since the robot is running). This proves the agent integration is functional.

3. **Implement Core Tools (Week 1-2):** Develop the movement and status tools in the UI repo:

4. **Movement**: Write `move_forward`, `move_backward`, `turn_angle`, `stop_robot` functions. Initially, for testing without a real robot, tie them into a simulator or simply log actions. If using a simulator (like turtlesim or Gazebo with a simple diff-drive robot), test that the robot moves as expected. Use rclpy to publish to `/turtle1/cmd_vel` (for turtlesim) as a test. Gradually adapt to actual Ranger topic names (likely `/cmd_vel` for the base, etc.).
5. **Status**: Implement a fake battery for testing (or use turtlesim pose as a "status"). Ensure agent can call these and respond.
6. Integrate these tools into the agent's tool list (replace or augment the default tools in ros-technician-cli). Likely, we'll create our agent by subclassing ROSA: e.g., `class RangerAgent(ROSA): tools = [MoveForwardTool,...]`. Otherwise, we use LangChain's `AgentExecutor` and give it our tools explicitly, as shown in the Gradio example[14].
7. *Milestone:* In simulation, user says "move forward 1 meter"; the agent selects MoveForward, our tool publishes the velocity, the simulated robot moves ~1 m, and the agent replies "Done.". Also user can ask "battery?" and get a made-up answer (to be replaced with real data when on hardware).

8. **Test on Real Hardware (Week 3):** Once basic tools are working in simulation, deploy the UI on the actual Ranger robot (or a development unit). Tune the parameters (speed, etc.) to the real robot's safe values. Test various commands in a controlled environment:

9. Short movements, rotations – verify accuracy and that STOP works if pressed mid-action.
10. Status queries – ensure correct values (battery percentage reading matches known value).
11. Nonsense or out-of-scope queries – agent should politely refuse or say it cannot do that (check prompt tuning).
12. Multi-step command – e.g., "turn 90 and move 0.5m": see if agent does both (it might not parse two actions in one sentence in MVP; if not, that's acceptable for now or we break it down).
13. Failure injection – e.g., turn off a ROS node and ask status to see if agent reports an issue.
14. *Milestone:* MVP acceptance test – operator uses only the chat UI to drive the robot around a bit and check status, without any crashes or unsafe behavior.

15. **User Confirmation & Refine Safety (Week 3-4):** Implement the confirm-before-execute for risky actions. This likely means detecting in the agent's plan if a tool is flagged as needing confirmation. Possibly simplest is to implement a wrapper tool that does:

    ```
    def move_distance_safe(dist):
        return "CONFIRM: Move %.1f m?" % dist
    ```

    and the agent would output that to user, then user must explicitly confirm which triggers the actual Move tool. Alternatively, integrate this into the UI logic: when agent is about to run MoveForward with a large distance, pause and ask. We'll decide the mechanism and implement it. Also adjust prompts to encourage the agent to ask for confirmation for large distances or unknown areas.

16. Additionally, incorporate any sensor safety check (if Ranger has a forward sonar, for instance, check it before moving).
17. *Milestone:* When user says "move 5 meters forward", the agent (or UI) responds with "I can move 5m forward. Please confirm." Only after yes, it proceeds.

18. **Enhanced UI Elements (Week 4):** Add a simple telemetry display on the Gradio interface outside the chat. For example, use `gr.Textbox` or `gr.Label` to show battery continuously (update every N seconds via a thread or via a Gradio `live=True` component). Add a manual control widget if possible – maybe a set of buttons or a small JS canvas for teleop. Gradio might not have a built-in joystick, but even arrow buttons that call a small function to publish Twist for a short burst would do. This runs in parallel with the chat agent.

19. *Milestone:* The UI now has at least a battery indicator that updates, and possibly arrow buttons that the user can click to drive the robot (useful if LLM is not needed for a simple maneuver).

20. **Documentation and Testing (Week 4):** Write comprehensive README and usage guide. Include how to set up API keys, how to switch to local model, and how to run the UI. Document safety guidelines for users (e.g., "Always supervise the robot. Emergency stop is available."). Perform testing of edge cases and finalize the prompt for the agent to ensure it's robust.

21. *Milestone:* Code freeze for MVP, all tests passing (including user acceptance tests by a few team members giving various commands).

22. **Post-MVP (Beyond 4 Weeks):** Start implementing the near-term features like navigation integration (this might involve bringing in Nav2 and mapping data, which is a bigger chunk), and iterate on the agent's intelligence (maybe fine-tune a local model on our specific command style to reduce reliance on OpenAI). Also gather feedback from initial users to improve the UI/UX (maybe the wording of responses, or adding synonyms for commands, etc.).

Throughout these steps, we plan to leverage existing examples and possibly even reuse code where available, rather than reinventing. For instance, if `ros-technician-cli` already has a "stop" tool or diagnostics tool, we integrate those. We keep the design agile so we can incorporate improvements from open-source projects doing similar things.

Reference Implementations and Inspirations

Our design is informed by several open-source projects and tools at the intersection of LLMs, robotics, and web interfaces. We draw on their ideas while tailoring the solution to Ranger's needs:

- **NASA JPL ROSA**[15][10] – *Robot Operating System Assistant* is an open-source ROS conversational agent. ROSA demonstrated the viability of using LangChain agents to query and command ROS systems using natural language. We reuse ROSA's core concept of an LLM-driven agent with a set of ROS tools. In fact, `ros-technician-cli` appears aligned with ROSA's approach. **Reuse:** The idea of inheriting a base agent and adding custom tools[16]. Potentially reuse some prompt tuning and tool implementations (like listing ROS topics, nodes). **Avoid:** ROSA is very general; we avoid any unnecessary complexity (e.g., multi-robot or ROS1 compatibility) that doesn't apply to our single ROS2 robot scenario.

- **Yutarop's TurtleBot3 Agent**[3][12] – This project provides a ROS2 Humble package where an LLM controls a TurtleBot3 in natural language. It splits functionalities into `motion_tools.py`, `status_tools.py`, etc., similar to our plan[3]. **Reuse:** The modular tool organization and examples of tool descriptions. Also, their approach to multi-backend LLM support (OpenAI, Anthropic, etc.) via environment config is something we emulate[5]. **Avoid:** It's tailored to TurtleBot3; Ranger has different capabilities. We won't copy any kinematic specifics. But the general patterns (e.g., how they use LangChain agents within a ROS2 node and use colcon to build) are highly relevant.

- **ATh0ft's Bimanual Robot Controller** (LangChain agent for a dual-arm robot)[17] – This is an academic project integrating ChatGPT with ROS2 for complex manipulation tasks. **Reuse:** It shows that even complex actions (bimanual) can be broken into LLM tools, reinforcing our decision to use an agent for multi-step tasks. We might glean how they handle confirmation or complex prompts. **Avoid:** Likely very specific to their hardware; also possibly not focused on safe autonomy. We stick to simpler mobile base commands first.

- **Robotec.AI RAI** (Robotics Agentic Infrastructure)[18] – A large framework for agentic robotics with ROS2, supporting complex scenarios, voice, logging, etc. **Reuse:** Conceptual inspiration for future features: RAI includes logging of actions, scenario definitions, and a voice interface. It confirms that our roadmap (voice input, scenario scripts, summarization) is feasible. **Avoid:** RAI is quite heavyweight (430+ stars, multi-agent focus) and likely would introduce a lot of abstraction. For our implementation-grade design, we favor a purpose-built approach for Ranger rather than adopting RAI directly (which might be overkill and less instructive for our team).

- **tessel-la Robo-Boy**[19] – A web application for controlling ROS2 robots via a gamepad interface, using rosbridge and a React frontend. **Reuse:** The concept of a **mobile-friendly UI with multiple control modes**. They even have a "voice command interface" tab[20][21], which is similar to what

we envision adding (ours using Gradio's speech recognition eventually). Also, their use of **rosbridge** shows how to connect a web UI to ROS; in our case, since we run locally, we avoid rosbridge, but if we ever needed remote access, we might integrate rosbridge. **Avoid:** Their architecture uses Docker, rosbridge, etc., which is different from our integrated approach. We also prefer Gradio for quick UI building instead of a custom React app (trading off some flexibility for speed). However, we might take UI ideas (like the dual-joystick layout for manual teleop).

- **ros2-web-bridge Teleop Demo (Hadabot)**[22] – An example of using Robot Web Tools (ros2-web-bridge + a webpage) to teleoperate a ROS2 robot from a browser. **Reuse:** Validates the approach of web-based control and demonstrates basic teleop functionality in a browser. Emphasizes the user experience of having everything accessible via a web UI. **Avoid:** Using ros2-web-bridge for local deployment (not needed for us). Also, that setup might have latency; since we are on robot, we keep it direct.

- **LangChain Gradio Agent Template**[1][2] – Examples from LangChain and Gradio documentation that show how to wire an agent's `AgentExecutor` to a Gradio chat UI with streaming. We follow this template closely to implement our streaming responses and the visualization of thought steps. **Reuse:** Code pattern for `interact_with_agent` coroutine and Chatbot UI updating with tool use. **Avoid:** Any reliance on cloud-specific features (our solution should run offline if needed), and we'll trim the example code to our needs (the example uses SERPAPI etc., which we don't).

- **Foxglove Studio & ROSBoard** – While not LLM-related, these are tools for robot telemetry visualization. **Reuse:** Inspiration for how to present telemetry (e.g., charts for battery over time, video feed, etc.). We might use Foxglove or ROSBoard alongside our UI for advanced monitoring. In design, we decided to include a minimal telemetry display ourselves for convenience, but we acknowledge these robust tools exist. **Avoid:** Trying to replicate full functionality of Foxglove; we only implement what's needed for our MVP (a lightweight view). Our focus is on command interface, not building a full dashboard from scratch (especially since Foxglove could be run in parallel if needed).

In summary, our approach combines the **agentic NLP capabilities of ROSA/TurtleBot3-agent**[3] with the **user-friendly web UI concepts of Robo-Boy and teleop demos**[19], all implemented in a way that adheres to ROS 2 best practices and Ranger's specific requirements. By studying these references, we ensure our design is grounded in proven ideas while avoiding pitfalls (like uncontrolled actions or overly complex setups). Each component we propose has precedent in the community, giving us confidence in feasibility.

---

[1] [2] [6] [7] [14] Agents And Tool Usage

https://www.gradio.app/guides/agents-and-tool-usage

[3] [5] [11] [12] GitHub - Yutarop/turtlebot3_agent: Control TurtleBot3 with natural language using LLMs

https://github.com/Yutarop/turtlebot3_agent

[4] About ROS 2 client libraries — ROS 2 Documentation: Foxy documentation

https://docs.ros.org/en/foxy/Concepts/About-ROS-2-Client-Libraries.html

[8] ROS 2 Package Structure: Python vs C++ Explained | Robotisim

https://robotisim.com/ros2-package-structure-python-vs-cpp/

[9] [13] Enabling Novel Mission Operations and Interactions with ROSA - arXiv

https://arxiv.org/html/2410.06472v2

[10] [15] [16] GitHub - nasa-jpl/rosa: ROSA is an AI Agent designed to interact with ROS1- and ROS2-based robotics systems using natural language queries. ROSA helps robot developers inspect, diagnose, understand, and operate robots.

https://github.com/nasa-jpl/rosa

[17] GitHub - ATh0ft/langchain_agent_robot_controller_ros2: An AI interface to control a bimanual robot using ChatGPT and LangChain

https://github.com/ATh0ft/langchain_agent_robot_controller_ros2

[18] GitHub - RobotecAI/rai: RAI is a vendor agnostic agentic framework for Physical AI robotics, utilizing ROS 2 tools to perform complex actions, defined scenarios, free interface execution, log summaries, voice interaction and more.

https://github.com/RobotecAI/rai

[19] [20] [21] GitHub - tessel-la/robo-boy: A web application for controlling ROS 2 robots, tailored for mobile usage. Inspired by retro handheld consoles.

https://github.com/tessel-la/robo-boy

[22] Launch a teleop controller for ROS2 in your web browser using ros2-web-bridge - Projects - Open Robotics Discourse

https://discourse.openrobotics.org/t/launch-a-teleop-controller-for-ros2-in-your-web-browser-using-ros2-web-bridge/13194